

- [Solution Architect — Questionnaire & Answers](#)
 - [Part 3: Docker, Kubernetes & CI/CD](#)
 - [Section 8: Docker](#)
 - [Q26. Explain Docker architecture and key concepts.](#)
 - [Q27. How do you optimize Docker images for production?](#)
 - [Q28. Explain Docker networking — bridge, host, overlay.](#)
 - [Section 9: Kubernetes \(EKS\)](#)
 - [Q29. Explain Kubernetes architecture and core components.](#)
 - [Q30. How do you set up EKS for production? Walk through your architecture.](#)
 - [Q31. Explain Kubernetes deployment strategies.](#)
 - [Q32. Explain IRSA \(IAM Roles for Service Accounts\) in EKS.](#)
 - [Section 10: CI/CD & DevOps](#)
 - [Q33. Design a CI/CD pipeline for a production application on EKS.](#)
 - [Q34. Explain GitOps with ArgoCD. How does it differ from traditional CI/CD?](#)
 - [Q35. What is Infrastructure as Code testing? How do you test Terraform code?](#)

Solution Architect — Questionnaire & Answers

Part 3: Docker, Kubernetes & CI/CD

Section 8: Docker

Q26. Explain Docker architecture and key concepts.

Answer:

Docker Architecture:

Docker Client (CLI) → Docker Daemon (dockerd) → Container Runtime
(containerd → runc)

↓
Images (read-only layers)
Containers (writable layer on top)
Volumes (persistent data)
Networks (bridge, host, overlay)

Key Concepts:

- **Image:** Immutable, layered filesystem template built from a Dockerfile
- **Container:** Running instance of an image with its own isolated namespace (PID, network, mount)
- **Dockerfile:** Build instructions — each instruction creates a layer
- **Registry:** Image storage (ECR, Docker Hub)
- **Volume:** Persistent storage that survives container restarts
- **Network:** Isolated communication channel between containers

Multi-Stage Dockerfile (Python example):

```
# Stage 1: Build/test
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --user -r requirements.txt
COPY . .
RUN python -m pytest tests/ --tb=short

# Stage 2: Production image
FROM python:3.11-slim
WORKDIR /app
COPY --from=builder /root/.local /root/.local
COPY --from=builder /app/src ./src
ENV PATH=/root/.local/bin:$PATH
USER 1000:1000
EXPOSE 8080
HEALTHCHECK --interval=30s CMD curl -f http://localhost:8080/health || exit
1
CMD ["python", "-m", "uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "8080"]
```

Best Practices:

- Use multi-stage builds (smaller images, no build tools in prod)

- Pin base image versions (`python:3.11.9-slim`, not `python:latest`)
 - Run as non-root user (`USER 1000:1000`)
 - Use `.dockerignore` to exclude `.git`, `__pycache__`, `.env`
 - One process per container
 - Use `HEALTHCHECK` for orchestrator integration
 - Scan images for CVEs: `trivy image myapp:latest`
-

Q27. How do you optimize Docker images for production?

Answer:

Technique	Impact	Example
Multi-stage build	50-90% size reduction	Separate build and runtime stages
Slim/Alpine base	80% smaller than full images	<code>python:3.11-slim</code> (150MB) vs <code>python:3.11</code> (900MB)
Layer ordering	Faster builds via cache	<code>COPY requirements.txt</code> first, then source code
No cache pip	Smaller layers	<code>pip install --no-cache-dir</code>
Combine RUN	Fewer layers	<code>RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*</code>
Distroless images	Minimal attack surface	<code>gcr.io/distroless/python3</code> (no shell, no package manager)

Image Scanning in CI/CD:

```
# GitLab CI
scan:
  stage: security
  script:
    - trivy image --exit-code 1 --severity HIGH,CRITICAL
$ECR_REPO:$CI_COMMIT_SHA
```

Q28. Explain Docker networking — bridge, host, overlay.

Answer:

Network Type	Scope	Use Case
Bridge	Single host	Default. Containers communicate via internal DNS on same host
Host	Single host	Container shares host's network stack. Best performance, no isolation
Overlay	Multi-host (Swarm/K8s)	Cross-host container communication
None	Isolated	No networking. For batch jobs needing full isolation

In Kubernetes (EKS): Docker networking is abstracted away. Kubernetes uses **CNI plugins** (AWS VPC CNI for EKS) that assign real VPC IPs to each pod, enabling direct pod-to-pod and pod-to-AWS-service communication.

Section 9: Kubernetes (EKS)

Q29. Explain Kubernetes architecture and core components.

Answer:

Control Plane (managed by AWS in EKS):

- └─ kube-apiserver → REST API, all communication goes through here
- └─ etcd → Distributed key-value store (cluster state)
- └─ kube-scheduler → Assigns pods to nodes based on resource requirements
- └─ kube-controller-manager → Reconciliation loops (ReplicaSet, Deployment, Node controllers)
- └─ cloud-controller-manager → AWS-specific (ELB, EBS, ENI)

Worker Nodes (you manage or use Fargate):

- └─ kubelet → Agent that ensures pods are running
- └─ kube-proxy → Network rules (iptables/IPVS) for service routing
- └─ Container Runtime → containerd (runs actual containers)

Core Objects I Use Daily:

Object	Purpose
Pod	Smallest deployable unit (1+ containers)
Deployment	Manages ReplicaSets, rolling updates, rollbacks
Service	Stable endpoint for pods (ClusterIP, NodePort, LoadBalancer)
Ingress	HTTP/HTTPS routing (ALB Ingress Controller in EKS)
ConfigMap	Non-sensitive configuration
Secret	Sensitive data (encrypted at rest with KMS in EKS)
PersistentVolumeClaim	Storage request (backed by EBS/EFS CSI driver)
HPA	Auto-scale pods based on metrics
PDB (PodDisruptionBudget)	Ensure minimum availability during disruptions
NetworkPolicy	Pod-level firewall rules
ServiceAccount	Pod identity (maps to IAM Role via IRSA)

Q30. How do you set up EKS for production? Walk through your architecture.

Answer:

Terraform-based EKS Setup:

VPC (3-AZ)

- └─ Public Subnets → ALB Ingress Controller, NAT Gateways
- └─ Private Subnets → EKS Worker Nodes (Karpenter-managed)
- └─ Data Subnets → RDS, ElastiCache

EKS Cluster

- └─ Control Plane → Managed by AWS, private API endpoint
- └─ Node Groups
 - └─ System: t3.medium (CoreDNS, kube-proxy, metrics-server)
 - └─ Karpenter: Provisions right-sized nodes on demand (Spot + On-Demand)
- └─ Add-ons
 - └─ AWS Load Balancer Controller (ALB Ingress)
 - └─ EBS CSI Driver (persistent volumes)
 - └─ Cluster Autoscaler or Karpenter
 - └─ Metrics Server (HPA)
 - └─ External DNS (Route53 integration)
 - └─ ArgoCD (GitOps deployments)
- └─ Security
 - └─ IRSA (IAM Roles for Service Accounts)
 - └─ OPA Gatekeeper (policy enforcement)
 - └─ Secrets encryption via KMS
 - └─ Private endpoint + Security Groups

Key Terraform Configuration:

```
module "eks" {
  source = "terraform-aws-modules/eks/aws"
  version = "~> 20.0"

  cluster_name      = "production"
  cluster_version   = "1.29"

  vpc_id            = module.networking.vpc_id
  subnet_ids        = module.networking.private_subnet_ids

  cluster_endpoint_public_access = false # Private API server
  cluster_endpoint_private_access = true

  # Enable secrets encryption
  cluster_encryption_config = {
    provider_key_arn = aws_kms_key.eks.arn
    resources        = ["secrets"]
  }

  # Managed node group for system components
  eks_managed_node_groups = {
    system = {
      instance_types = ["t3.medium"]
    }
  }
}
```

```
    min_size      = 2
    max_size      = 4
    desired_size  = 2
    capacity_type = "ON_DEMAND"
  }
}
```

Q31. Explain Kubernetes deployment strategies.

Answer:

Strategy	How It Works	Risk	Use Case
Rolling Update	Replace pods gradually (default)	Brief mixed versions	Standard deployments
Blue/Green	Deploy new version alongside old, switch traffic	Double resources temporarily	Zero-downtime, instant rollback
Canary	Route small % of traffic to new version	Complexity	High-risk changes, gradual validation
Recreate	Kill all old pods, create new ones	Downtime	Dev/test, database migrations

Rolling Update Config:

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 25%      # Max extra pods during update
      maxUnavailable: 0  # Zero downtime
      minReadySeconds: 30 # Wait before marking pod as ready
```

Canary with ArgoCD + Argo Rollouts:

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
```

```
spec:
  strategy:
    canary:
      steps:
        - setWeight: 5          # 5% traffic to canary
        - pause: { duration: 5m }
        - setWeight: 20
        - pause: { duration: 10m }
        - setWeight: 50
        - pause: { duration: 10m }
      canaryService: myapp-canary
      stableService: myapp-stable
```

Q32. Explain IRSA (IAM Roles for Service Accounts) in EKS.

Answer:

Problem: How do pods access AWS services (S3, DynamoDB, Bedrock) securely?

Old way: Node-level IAM role → Every pod on the node gets the same permissions (violates least privilege)

IRSA (correct way): Map a Kubernetes ServiceAccount to a specific IAM Role

```
Pod → K8s ServiceAccount → OIDC Trust → AWS IAM Role → Fine-grained
permissions
```

Setup:

```
# 1. Enable OIDC provider for EKS
resource "aws_iam_openid_connect_provider" "eks" {
  url            = module.eks.cluster_oidc_issuer_url
  client_id_list = ["sts.amazonaws.com"]
  thumbprint_list =
[data.tls_certificate.eks.certificates[0].sha1_fingerprint]
}

# 2. Create IAM role with OIDC trust
resource "aws_iam_role" "app_role" {
  name = "app-pod-role"
  assume_role_policy = jsonencode({
    Statement = [{
      Effect = "Allow"
```



```

Principal = { Federated = aws_iam_openid_connect_provider.eks.arn }
Action = "sts:AssumeRoleWithWebIdentity"
Condition = {
    StringEquals = {
        "${replace(module.eks.cluster_oidc_issuer_url, "https://",
""}):sub" = "system:serviceaccount:default:app-sa"
    }
}
}]
})
}

# 3. Annotate K8s ServiceAccount
resource "kubernetes_service_account" "app" {
    metadata {
        name      = "app-sa"
        namespace = "default"
        annotations = {
            "eks.amazonaws.com/role-arn" = aws_iam_role.app_role.arn
        }
    }
}
}

```

Result: Only pods using **app-sa** ServiceAccount get the specific IAM permissions. All other pods have zero AWS access.

Section 10: CI/CD & DevOps

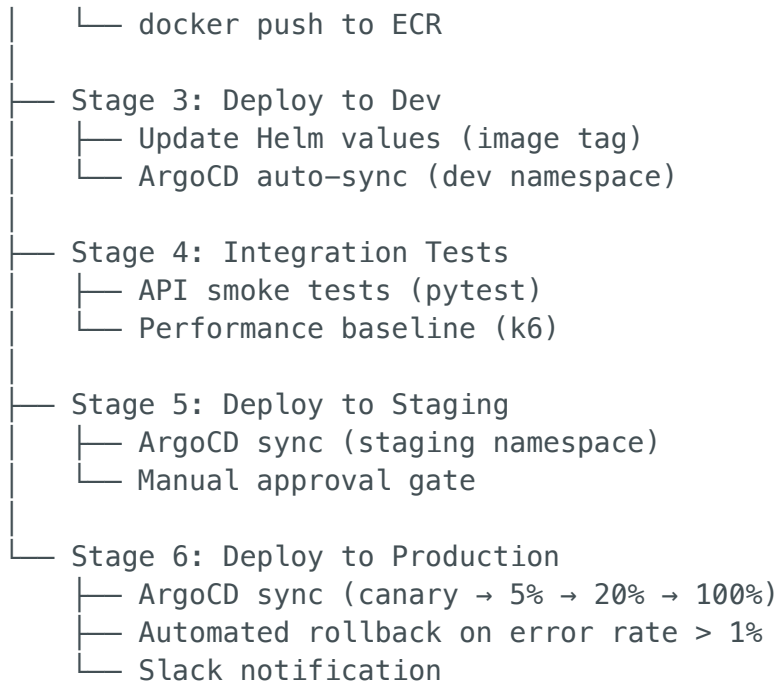
Q33. Design a CI/CD pipeline for a production application on EKS.

Answer:

```

Developer → Git Push → GitLab CI/CD
├── Stage 1: Lint & Test
│   ├── terraform fmt -check
│   ├── terraform validate
│   ├── tflint
│   ├── pytest (unit + integration)
│   └── checkov (security scan for IaC)
└── Stage 2: Build & Scan
    ├── docker build (multi-stage)
    └── trivy image scan (block on CRITICAL)

```



GitLab CI Example:

```
stages:
  - test
  - build
  - deploy-dev
  - deploy-prod

test:
  stage: test
  image: python:3.11-slim
  script:
    - pip install -r requirements.txt
    - pytest tests/ --cov=src --cov-report=term-missing
    - checkov -d terraform/ --framework terraform

build:
  stage: build
  services:
    - docker:dind
  script:
    - aws ecr get-login-password | docker login --username AWS --password-
stdin $ECR_REGISTRY
    - docker build -t $ECR_REPO:$CI_COMMIT_SHA .
    - trivy image --exit-code 1 --severity CRITICAL $ECR_REPO:$CI_COMMIT_SHA
    - docker push $ECR_REPO:$CI_COMMIT_SHA

deploy-prod:
  stage: deploy-prod
  when: manual # Require approval
  script:
    - yq eval ".image.tag = \"${CI_COMMIT_SHA}\"" -i helm/values-prod.yaml
    - git add . && git commit -m "deploy: ${CI_COMMIT_SHA}" && git push
    # ArgoCD detects Git change and syncs automatically
```

Q34. Explain GitOps with ArgoCD. How does it differ from traditional CI/CD?

Answer:

Traditional CI/CD (Push Model):

```
CI pipeline → builds artifact → pushes to cluster (kubectl apply / helm upgrade)
```

- Pipeline has cluster credentials
- No single source of truth
- Drift possible (manual kubectl changes)

GitOps (Pull Model):

```
CI pipeline → builds artifact → updates Git repo (image tag)  
ArgoCD (in-cluster) → watches Git → pulls and syncs changes
```

GitOps Principles:

1. **Git is the single source of truth** for desired state
2. **Declarative configuration** (Helm charts / Kustomize / plain YAML)
3. **Automated reconciliation** — ArgoCD continuously compares Git vs cluster state
4. **Drift detection** — Any manual **kubectl** change is detected and auto-corrected

ArgoCD Application CRD:

```
apiVersion: argoproj.io/v1alpha1  
kind: Application  
metadata:  
  name: myapp  
  namespace: argocd  
spec:  
  project: default  
  source:  
    repoURL: https://github.com/myorg/k8s-manifests.git  
    targetRevision: main  
    path: apps/myapp  
    helm:
```

```
valueFiles:
  - values-prod.yaml
destination:
  server: https://kubernetes.default.svc
  namespace: production
syncPolicy:
  automated:
    prune: true      # Delete resources removed from Git
    selfHeal: true   # Revert manual changes
syncOptions:
  - CreateNamespace=true
```

Benefits:

- **Audit trail:** Every change is a Git commit (who, what, when, why)
- **Rollback:** `git revert` instantly rolls back the deployment
- **Security:** CI pipeline never touches the cluster — only ArgoCD (in-cluster) has access
- **Drift prevention:** `selfHeal: true` reverts any manual changes

Q35. What is Infrastructure as Code testing? How do you test Terraform code?

Answer:

Test Type	Tool	What It Tests
Formatting	<code>terraform fmt -check</code>	Consistent code style
Validation	<code>terraform validate</code>	Syntax and internal consistency
Linting	<code>tflint</code>	AWS-specific issues (invalid instance types, deprecated resources)
Security	<code>checkov, tfsec</code>	Misconfigurations (public S3, unencrypted RDS, open security groups)
Plan	<code>terraform plan</code>	Preview changes before applying
Unit Test	<code>terraform test</code> (built-in)	Validate module logic with mock providers

Test Type	Tool	What It Tests
Integration	terratest (Go)	Deploy real infra, validate, destroy
Policy	OPA/Sentinel	Enforce organizational policies

Checkov Example:

```
$ checkov -d terraform/ --framework terraform

Passed: 45
Failed: 3
- CKV_AWS_18: "Ensure S3 bucket has access logging enabled"
- CKV_AWS_145: "Ensure S3 bucket is encrypted with KMS"
- CKV_AWS_144: "Ensure S3 bucket has cross-region replication enabled"
```

CI Pipeline Integration:

```
terraform-checks:
  script:
    - terraform fmt -check -recursive
    - terraform init -backend=false
    - terraform validate
    - tflint --recursive
    - checkov -d . --framework terraform --compact --quiet
```

This concludes Part 3. Continue to Part 4 for Python Testing, Web Architecture Design, AI/ML, and Scenario-Based Questions.